

A Global Neural Network Model for the Fast Approximation of Green’s Function for the Hydrodynamic Simulation of Large Offshore Systems

Aitzaz Munir Shaikh

Abstract

This paper presents the development and testing of a physics-informed neural network (PINN) aimed at approximating the Green’s function solution used in hydrodynamic simulations of large offshore systems. Green’s function, while theoretically solvable, poses significant computational challenges in practice. Using a combination of machine learning techniques and physics-based modeling, we replicated and attempted to surpass benchmark results from a seminal paper (Zhan et al.). Our proposed improvements include training a global network across all domains, implementing rational activation functions, and refining the training data distribution to emphasize accuracy in high-variability regions.

1 Introduction

The Boundary Element Method (BEM) is the most commonly used numerical method to solve the three-dimensional diffraction-radiation problem of floating bodies. BEMs usually require the solution of the PDE for appropriate boundary conditions, which is then transformed into boundary integral equations using Green’s Theorem. This is still an active area of research.

The challenge with many expressions of Green’s Functions is that they tend to scale as $\mathcal{O}(N^2)$ with the number of panels, which is problematic especially when these traditional simulation methods are used with large-scale optimization for offshore structures involving many panels and parameters. Therefore, a trade-off between accuracy and efficiency should be considered.

We propose a hybrid approach of using neural networks to enhance the traditional BEM program. The result of augmentation is that the traditional program is differentiable, numerically efficient, and differentiable in most cases. This will significantly reduce time per optimization iteration for large-scale optimization.

2 Oscillatory Green's Function

Green's function is defined using the following mathematical expression:

$$G = \frac{1}{k_0 \sqrt{X^2 + (z_1 - z_2)^2}} + \frac{1}{\sqrt{X^2 + Y^2}} + F(X, Y) + 2\pi i k_0 e^{-Y} J_0(X) \quad (1)$$

The two nondimensional coordinates are defined to be radial and vertical separation between the field point and the image source point above the free surface.

$$X = k_0 \sqrt{(x_1 - x_2)^2 + (y_1 - y_2)^2}, \quad Y = -k_0(z_1 + z_2)$$

The term $F(X, Y)$ is defined as:

$$F(X, Y) = \frac{1}{\pi} \int_0^\infty dk \int_0^{2\pi} d\theta \frac{1}{k-1} \cdot e^{k(-Y+iX \cos \theta)} \quad (2)$$

However, this poses several computational challenges due to the pole in the integrand. The integral is transformed to a finite, non-singular form. The derivation by J.N. Newman, which allows for highly accurate numerical evaluation using standard quadrature methods, is expressed as:

$$F(X, Y) = -2e^{-Y} \int_0^Y \frac{e^t}{\sqrt{X^2 + t^2}} dt - \pi e^{-Y} [B(X) + H(X)] \quad (3)$$

where $B(x)$ is the 0th order Bessel function of the second kind and $H(x)$ is the 0th order Struve function.

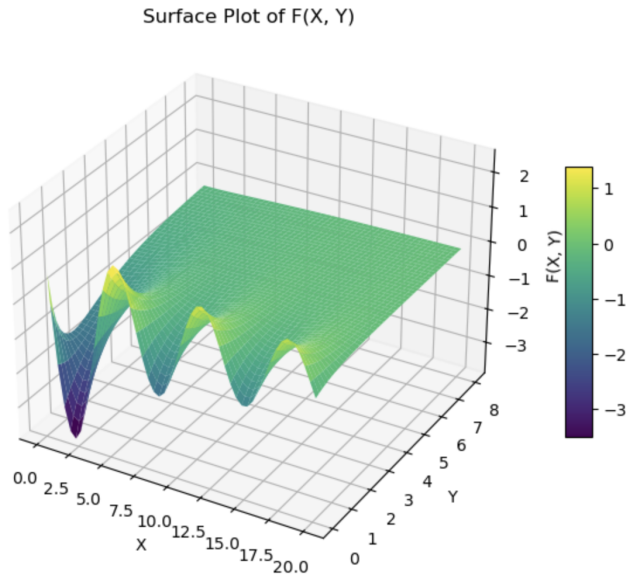


Figure 1: Plot of the Green's Function $G(X, Y)$ over the defined domain. This visualization is used as the qualitative reference throughout this paper.

As shown in Figure 1, the plot illustrates the oscillatory and singular behavior of the Green’s function, particularly near the origin. This reference will be used for qualitative comparison with the neural network predictions.

3 Previous Work

The paper introduces *StripeGF*, a novel machine learning framework designed to efficiently and accurately approximate the free-surface Green’s function, which is critical in potential flow-based hydrodynamic simulations. The method employs a two-part architecture:

- In **Domain 2** ($X > 4$), a single-input MLP learns the Green’s function along horizontal datum lines, while a fourth-order Runge–Kutta (RK4) solver interpolates between those lines.
- Near singularities in **Domain 1** ($X \leq 4$), a double-input MLP is employed to capture the more complex behavior.

The model is trained using high-precision data from Romberg integration and is structured to incorporate derivative information. *StripeGF* achieves over four-digit accuracy in 99.9% of the domain. It significantly outperforms traditional solvers in both speed and precision, and its effectiveness is validated through integration into a boundary element simulation of a container ship hull.

While this framework is highly accurate, it still poses numerical challenges, particularly in domain discretization and numerical considerations such as step size. Additionally, the requirement to train multiple multilayer perceptrons in Domain 2 increases the complexity of the training process.

Our work builds upon this foundation, particularly by adopting the Levenberg–Marquardt optimizer, which we found to significantly outperform other optimization strategies in our experiments.

4 Machine Learning Model

We employ a classic neural network approach. Through our experiments, we observed that changing the domain significantly affected the training time, accuracy, and required layer complexity. Below is a summary of how the neural network operates:

A neural network is a function approximator composed of layers of interconnected computational units, or *neurons*, designed to learn complex mappings from input to output via optimization. Each neuron performs a weighted sum of its inputs followed by a nonlinear activation function, such as the hyperbolic tangent ($\tanh$) or ReLU.

Mathematically, for an input vector $\mathbf{x}$, a single layer computes:

$$\mathbf{h} = \sigma(\mathbf{W}\mathbf{x} + \mathbf{b})$$

where $\mathbf{W}$ and $\mathbf{b}$ are the learnable weights and biases, respectively, and σ is the activation function. Layers are stacked to form a deep neural network, enabling hierarchical feature extraction.

The network parameters are trained by minimizing a loss function—typically the mean squared error (MSE) for regression tasks—using optimization algorithms such as gradient descent. The training process consists of:

- **Forward propagation** to compute predictions.
- **Backward propagation** to calculate gradients using the chain rule.
- **Parameter updates** to iteratively reduce the loss over the training dataset.

4.1 Training and Test Data

The training and test data were generated numerically by evaluating the Green’s function over a structured grid of spatial coordinates (X, Y) across the domain. To efficiently capture the function’s varying behavior, the domain was split into three regions, each with different sampling densities:

- **Region 1 (Very Fine Grid):** $X \in [0.1, 5.0]$, $Y \in [0.1, 4.0]$
Sampling interval: $\Delta X = \Delta Y = 0.02$ Number of points: **48,216**
- **Region 2 (Moderately Fine Grid):** $X \in [5.0, 20.0]$, $Y \in [0.1, 4.0]$
Sampling interval: $\Delta X = \Delta Y = 0.05$ Number of points: **23,858**
- **Region 3 (Coarse Grid):** $X \in [0.00001, 20.0]$, $Y \in [4.0, 8.0]$
Sampling interval: $\Delta X = \Delta Y = 0.2$ Number of points: **2,121**

This adaptive sampling strategy was designed to allocate more points to regions of the domain exhibiting sharp changes—particularly near the singularity at $(X, Y) = (0, 0)$ —and fewer points in smoother, planar regions. The resulting dataset contains a total of **74,195** coordinate pairs.

The dataset was saved in CSV format, randomly shuffled, and split into training and validation sets using an 85:15 ratio. All input features (X, Y) were normalized to the $[0, 1]$ range using MinMax scaling, while output values were left unscaled to retain physical meaning.

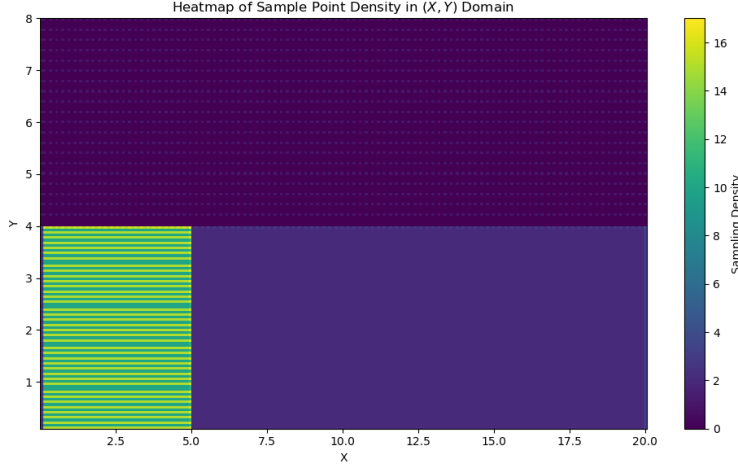


Figure 2: Heatmap of sampling density across the (X, Y) domain. The highest density of data points occurs near the origin ($X < 5, Y < 4$) to capture sharp variations in the Green’s function. Moderate density covers $X > 5, Y < 4$, and a coarser grid is used for the smoother region $Y > 4$.

Figure 2 provides a heatmap visualization of the point density across the domain. The dense clustering of points in the lower-left corner highlights the focus on regions where the Green’s function is most irregular.

4.2 Activation Function

We trained our models using the tanh activation function, inspired by the architecture described in Zhan et al. We also tested a variety of other functions, including rational activation functions. While rational activations showed some promise, tanh consistently outperformed all alternatives when used with the Levenberg–Marquardt (LM) optimizer. These results align with the empirical findings of Zhan et al.

4.3 Loss Function

We used the mean squared error (MSE) loss function, defined as:

$$\mathcal{L} = \frac{1}{N} \sum_{i=1}^N (f_{\theta}(\mathbf{x}_i) - y_i)^2$$

where f_{θ} denotes the network output, $\mathbf{x}_i$ the input, and y_i the ground truth. This choice is standard for regression problems and directly minimizes the discrepancy between the predicted and true values of the Green’s function.

4.4 Optimizer

Among all optimizers tested—including Adam, stochastic gradient descent (SGD), and AdaGrad—the Levenberg–Marquardt (LM) optimizer significantly outperformed the rest in terms

of final loss. Although training with LM required longer runtimes, it achieved a minimum loss of approximately 2.4×10^{-5} .

The additional computational cost was justified by the substantial gains in accuracy. Moreover, because our task focuses on precise function approximation rather than generalization to unseen data, overfitting is not a concern—in fact, it is beneficial in this context. The model’s ability to memorize the training distribution leads to better accuracy on the complex Green’s function surface.

5 Training Details and Results

5.1 Training Convergence and Model Architecture

Our model achieved strong convergence, attaining a final mean squared error (MSE) of approximately **0.000024**. The optimal performance was obtained using a neural network with the following architecture:

$$\text{layers} = [2, 16, 16, 8, 8, 1]$$

Here, the input layer has dimension 2 (corresponding to (X, Y)), and the output is a scalar representing $F(X, Y)$.

Training time was highly sensitive to the number of neurons per layer and the choice of optimizer. While optimizers like Adam and SGD exhibited slower and less precise convergence, the Levenberg–Marquardt (LM) optimizer—despite higher computational cost—consistently achieved significantly lower loss values. This optimizer was implemented via the `Optimistix` library in JAX.

We trained the model for 1316 epochs, with a maximum cap of 1500 epochs. The relative and absolute tolerances were both set to $1\text{e-}10$ and passed as parameters to the LM solver.

The training loss curve, shown in Figure 3, demonstrates smooth and stable convergence over time.

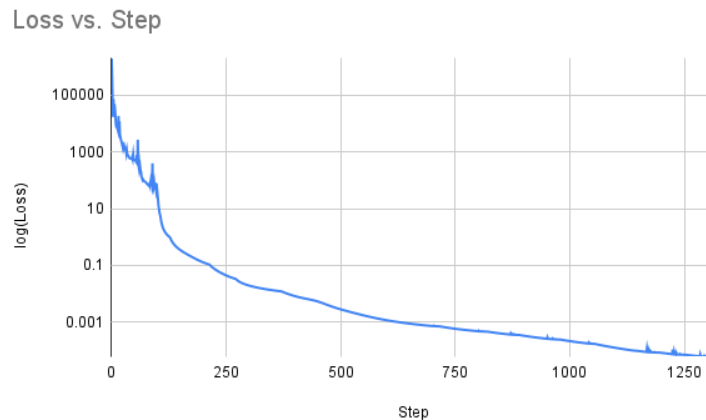


Figure 3: Training loss curve showing convergence of the model to an MSE of 0.00024 over 1316 epochs.

5.2 Qualitative Accuracy

Below is the final predicted Green's Function surface generated solely from the trained model's outputs. This can be directly compared to the exact solution graph presented earlier for qualitative assessment.

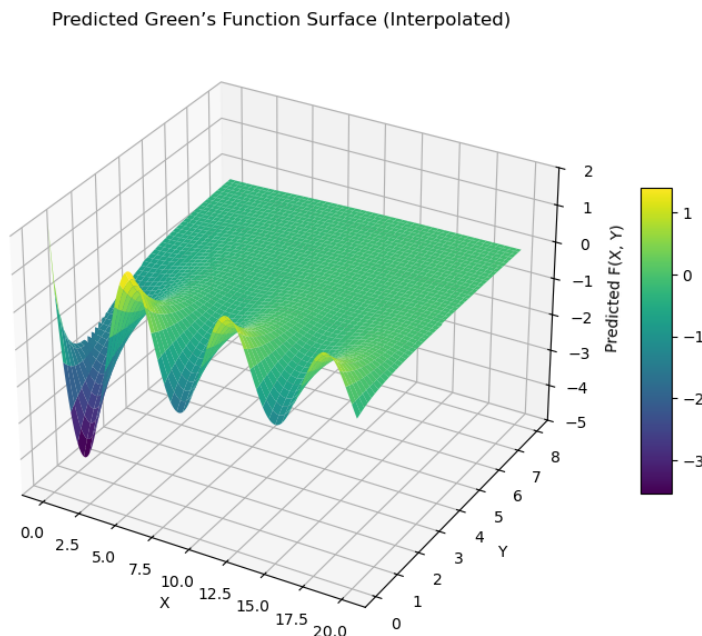


Figure 4: Predicted Green's Function surface using the trained model.

Additionally, we present the error distribution across three domains. Notably, in Domain A—located near the singularity—over **95%** of the test points achieved an accuracy of at least **four decimal places**. The error further decreases across Domains B and C, indicating the model's ability to generalize and capture oscillatory behavior even in regions with fewer data points.

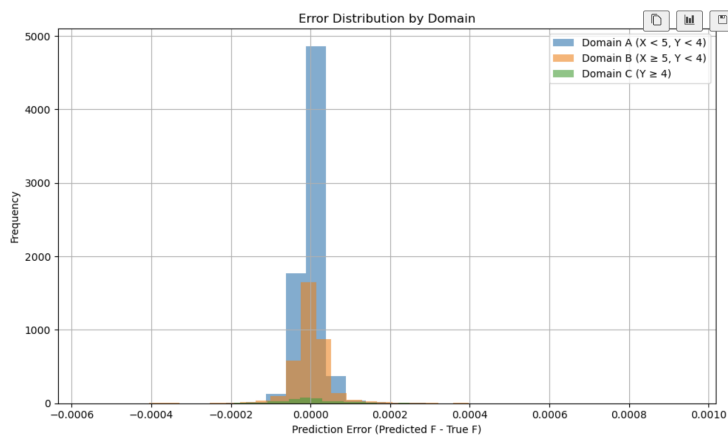


Figure 5: Distribution of absolute error across Domains A, B, and C.

To further illustrate the spatial structure of the model’s approximation error, we include a 3D error surface plot showing the difference $E_1 = \hat{F} - F$ over the entire domain. As seen below, the prediction errors are generally below 10^{-4} and localized spikes primarily occur near the singularity and along oscillatory regions.

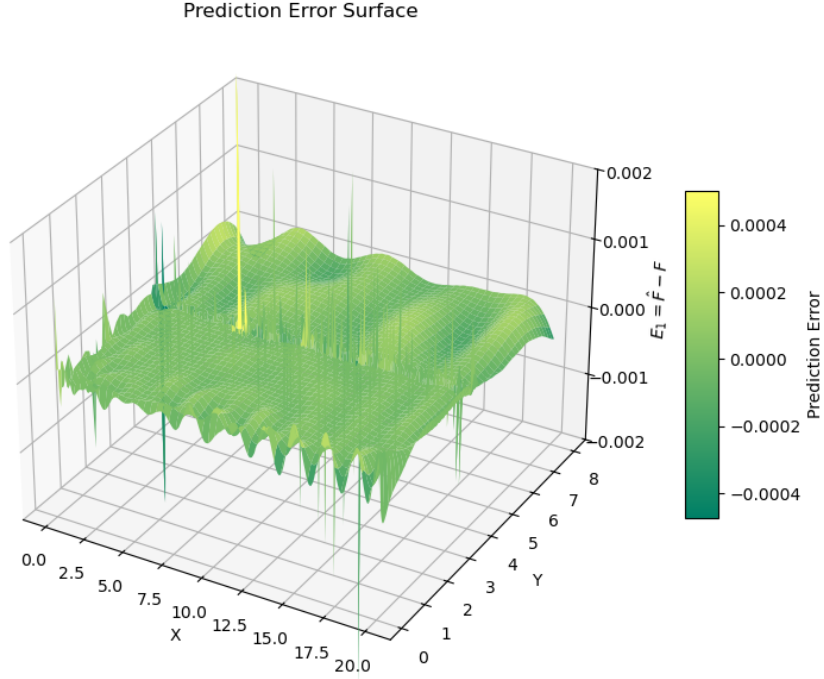


Figure 6: Prediction error surface $E_1 = \hat{F} - F$ across the (X, Y) domain. While minor spikes appear near the singularity, the error remains consistently low throughout.

5.3 Quantitative Accuracy

To evaluate the performance of our trained model, we computed several regression metrics over the validation set. The results confirm both high precision and reliability of the neural network in approximating the Green’s function.

- **Mean Absolute Error (MAE):** 0.000024
- **Mean Squared Error (MSE):** ≈ 0
- **R^2 Score (Explained Variance):** 1.000000
- **Custom Accuracy (within 10% tolerance):** 99.99%

These results indicate that the model’s predictions are nearly indistinguishable from the true values across the domain. The R^2 score of 1.0 reflects perfect explained variance, and the near-zero MSE confirms the model’s ability to replicate the Green’s function with remarkable precision.

6 Future Work

One potential direction is to investigate the relationship between the distance from the origin (i.e., proximity to the singularity) and the complexity of the neural network architecture—such as the number of hidden layers or neurons. A more expressive architecture may be necessary in regions with steeper gradients or oscillatory behavior.

We also propose exploring ensemble learning methods, such as bagging or boosting, to further improve accuracy and enhance the model’s ability to capture fine-grained local behavior.

While our current model achieves high global accuracy, it exhibits limitations in capturing localized oscillatory patterns. As shown in Figure 7, regions roughly along the line $5Y = X + 4$ show elevated residuals, suggesting difficulty in learning gradients and maintaining smoothness, whereas most other regions are accurately approximated.

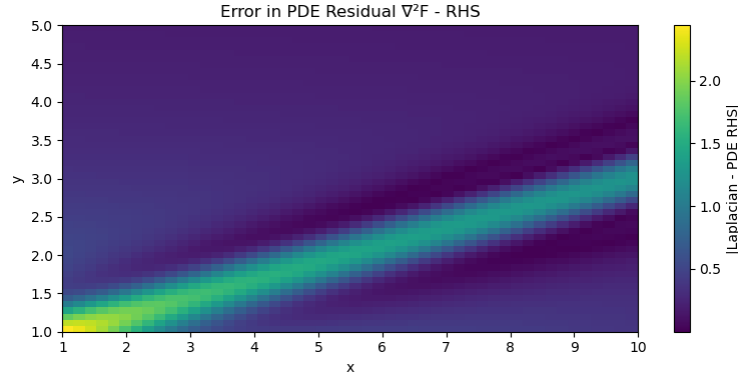


Figure 7: Error surface highlighting regions of high prediction error near the singularity and oscillatory zones.

To address this limitation, future work will involve incorporating the PDE residual directly into the loss function—effectively turning the model into a *Physics-Informed Neural Network (PINN)*. This approach constrains the solution space by enforcing consistency with the governing equation and could improve the model’s ability to generalize while retaining the physical structure of the Green’s function.

6.1 Updated Work: Branched Neural Network Framework

In our initial experiments, we encountered challenges when jointly minimizing both data loss (i.e., matching the Green’s function values) and gradient loss (i.e., enforcing derivative consistency). Adjustments to network parameters that improved one objective often worsened the other, leading to unstable optimization. To address this, we proposed a branched neural network framework that decouples these competing objectives.

6.1.1 Mathematical Framework

Let the base network be denoted by

$$h_{\text{base}}(\mathbf{x}; \theta) : \mathbb{R}^2 \rightarrow \mathbb{R},$$

where $\mathbf{x} = (X, Y)$ are the inputs and θ are the shared parameters.

The data and gradient losses are defined as

$$\begin{aligned}\mathcal{L}_{\text{data}}(\theta) &= \frac{1}{N} \sum_{i=1}^N \left(h_{\text{base}}(\mathbf{x}_i; \theta) - f(\mathbf{x}_i) \right)^2, \\ \mathcal{L}_{\text{grad}}(\theta) &= \frac{1}{N} \sum_{i=1}^N \left\| \nabla h_{\text{base}}(\mathbf{x}_i; \theta) - \nabla f(\mathbf{x}_i) \right\|^2.\end{aligned}$$

The base network is first trained with a joint objective

$$\mathcal{L}_{\text{joint}}(\theta) = \alpha \mathcal{L}_{\text{data}}(\theta) + \beta \mathcal{L}_{\text{grad}}(\theta),$$

optimized with the Levenberg–Marquardt method to moderate tolerance. This produces a parameter set θ^* that captures the global structure of the Green’s function.

From θ^* , we then branch into two refinements:

$$\theta_d^* = \arg \min_{\theta_d} \mathcal{L}_{\text{data}}(\theta_d), \quad \theta_g^* = \arg \min_{\theta_g} \mathcal{L}_{\text{grad}}(\theta_g),$$

yielding a data-focused model h_{data} and a gradient-focused model h_{grad} .

6.1.2 Inference with New Inputs

For a new input $\mathbf{x}_{\text{new}} = (X_{\text{new}}, Y_{\text{new}})$:

$$\hat{f}_{\text{data}} = h_{\text{data}}(\mathbf{x}_{\text{new}}; \theta_d^*), \quad \hat{f}_{\text{grad}} = h_{\text{grad}}(\mathbf{x}_{\text{new}}; \theta_g^*).$$

The final prediction may be formed as

$$\hat{f}(\mathbf{x}_{\text{new}}) = \lambda \hat{f}_{\text{data}} + (1 - \lambda) \hat{f}_{\text{grad}}, \quad \lambda \in [0, 1],$$

where λ controls the trade-off between value fidelity and derivative accuracy.

6.1.3 Experimental Results

The branched framework was tested across both small controlled domains and the full hydrodynamic domain.

- **Small domains:** The framework achieved the lowest overall loss of 2.7×10^{-5} after 4000 epochs, with data loss $\approx 1.2 \times 10^{-5}$ and gradient loss $\approx 5.0 \times 10^{-6}$. In terms of prediction accuracy, the best run produced RMSE of 4.30×10^{-4} (function values) and 8.13×10^{-4} (gradients), demonstrating four-decimal precision.

- **Branch size comparison:** Increasing branch width improved gradient accuracy significantly, reducing gradient RMSE from 8.45×10^{-3} to 4.05×10^{-3} , though function-value RMSE increased slightly from 3.94×10^{-3} to 5.26×10^{-3} . This confirmed the trade-off between data and gradient optimization.
- **Large physical domain:** On the full $x \in [0.1, 20], y \in [0.1, 8]$ domain (65,536 points), the network was trained for 4800 epochs. The final total loss reached 4.27×10^{-4} , with data loss $\approx 6.1 \times 10^{-5}$, gradient loss $\approx 2.24 \times 10^{-4}$, and shared loss $\approx 1.42 \times 10^{-4}$. Mean absolute errors were 5.45×10^{-3} (function values) and 5.35×10^{-3} (gradients), showing stable convergence at scale.

Summary. These experiments demonstrate that our proposed branched neural network framework achieves strong accuracy across domains. On smaller domains, the method reaches four-decimal precision for both function values and gradients. On larger domains, the framework converges reliably with mean errors on the order of 10^{-3} – 10^{-2} . Enlarging the branches confirms that gradient learning benefits most from specialization, supporting the value of the branched design.

7 Conclusion

We showed that a data-centric global neural network can accurately approximate the oscillatory Green’s function used in hydrodynamic simulations. Using the Levenberg–Marquardt optimizer and a sampling scheme biased toward regions of high variation, the original (single-network) model achieved very high fidelity on the target surface (final MSE $\approx 2.4 \times 10^{-5}$, $R^2 = 1.0$), confirming that careful data design can rival architectural complexity.

Building on this, we introduced an updated *branched* framework: a shared base network is trained to moderate tolerance on a joint value+gradient objective and then split into two heads that specialize separately in function values and gradients. In small, controlled domains this approach reached four-decimal precision on both quantities (with losses on the order of 10^{-5} and RMSEs in the 10^{-4} range). On the full hydrodynamic domain, it converged stably with mean errors in the 10^{-3} – 10^{-2} range, demonstrating practical scalability. Ablations further showed the intended trade-off: widening the branches improves gradient fidelity noticeably, with a modest cost to function-value accuracy, evidence that decoupling the objectives reduces the “tug-of-war” seen in joint training.

Overall, the results support two conclusions: (i) a data-centric strategy is an effective baseline for fast, accurate Green’s function approximation; and (ii) the branched framework is a successful extension that stabilizes training and improves gradient quality while preserving strong value accuracy. Remaining hotspots near narrow oscillatory regions suggest clear next steps: incorporate PDE residuals directly (PINN regularization), explore input-dependent weighting between branches, and tune model capacity as a function of proximity to the singularity to push toward uniform four-digit accuracy across the full domain.

References

- [1] Zhan, Ke, Renchuan Zhu, and Dekang Xu. *A machine learning model for fast approximation of free-surface Green's function and its application*. Journal of Ocean Engineering and Science, 2023.
- [2] Khanal, Kapil. *Machine Learning Accelerated Boundary Element Method for Hydrodynamic Simulation of Large Offshore Systems*, 20 Sept. 2024.